

МИНИСТЕРСТВО ТРАНСПОРТА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ АГЕНТСТВО ЖЕЛЕЗНОДОРОЖНОГО ТРАНСПОРТА
Государственное бюджетное образовательное учреждение
высшего образования
**«ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПУТЕЙ СООБЩЕНИЯ ИМПЕРАТОРА АЛЕКСАНДРА I»**

Кафедра «ИНФОРМАЦИОННЫЕ И ВЫЧИСЛИТЕЛЬНЫЕ СИСТЕМЫ»

Дисциплина: «Программирование Ассемблер»

О Т Ч Е Т
по лабораторной работе № 4

Выполнил студент
Факультета АИТ
Группы ИВБ-515
Принял

Нартов С. А.

Кукин М. Ю.

Санкт-Петербург
2026

Задача

На базе выполненной работы 3.2 изменить структуру программы таким образом, чтобы:

- формирование массива (генерация случайных элементов) была обособлена в макроопределение (макрос) с тремя параметрами – максимальное значение генерируемого случайного числа, длина массива (равна количеству генерируемых случайных чисел) и адрес первого элемента;
- вывод массива был организован в виде процедуры, получающей 2 параметра – длину массива и адрес первого элемента;
- сортировка массива была организована в виде внешней процедуры из самостоятельно созданной библиотеки. Параметры процедуры – длина массива и адрес начального элемента.

Листинг

```

.586p
.model flat, stdcall
option casemap: none

include C:\masm32\include\windows.inc
include C:\masm32\include\kernel32.inc
includelib C:\masm32\lib\kernel32.lib
include C:\masm32\include\user32.inc
includelib C:\masm32\lib\user32.lib
include C:\masm32\include\masm32.inc
includelib C:\masm32\lib\masm32.lib
include C:\masm32\include\msvcrt.inc
includelib C:\masm32\lib\msvcrt.lib
include C:\masm32\macros\macros.asm

includelib mylib.lib

shaker_sort PROTO :DWORD, :DWORD

GENERATE_ARRAY MACRO maxVal, arrLen, arrAddr
    LOCAL gen_loop, gen_char, rdrand_retry

    mov ecx, arrLen
    mov esi, arrAddr

gen_loop:
    push ecx
    push esi

    lea edi, rand_buf
    mov ecx, 4

gen_char:
    push ecx

rdrand_retry:
    db 0Fh, 0C7h, 0F0h
    jnc rdrand_retry

    xor edx, edx
    mov ecx, 94
    div ecx
    add edx, 33

    mov al, dl
    stosb

    pop ecx
    loop gen_char

```

```

    mov byte ptr [edi], 0

    invoke atodw, addr rand_buf

    xor edx, edx
    mov ecx, maxVal
    inc ecx
    div ecx
    mov eax, edx

    pop esi
    mov [esi], eax
    add esi, 4

    pop ecx
    loop gen_loop
ENDM

.data
    arr            dd 20 dup(?)
    arrSize        dd 20
    maxValue       dd 420

    rand_buf       db 5 dup(?)

    str_original    db "Original array:", 13, 10, 0
    str_sorted      db 13, 10, "Sorted array:", 13, 10, 0
    str_format_num  db "%d ", 0
    str_format_last db "%d", 13, 10, 0
    str_newline     db 13, 10, 0

.code
print_array PROC uses ecx esi, arrLen:DWORD, arrAddr:DWORD
    mov ecx, arrLen
    mov esi, arrAddr

print_loop_func:
    push ecx

    mov eax, [esi]

    cmp ecx, 1
    jg print_space_func

    invoke crt_printf, addr str_format_last, eax
    jmp print_next_func

print_space_func:
    invoke crt_printf, addr str_format_num, eax

print_next_func:

```

```

    add esi, 4
    pop ecx
    loop print_loop_func

    ret
print_array ENDP

main PROC
    GENERATE_ARRAY maxValue, arrSize, offset arr

    invoke crt_printf, addr str_original
    invoke print_array, arrSize, offset arr

    invoke shaker_sort, arrSize, offset arr

    invoke crt_printf, addr str_sorted
    invoke print_array, arrSize, offset arr
    invoke crt_printf, addr str_newline

    inkey "Press any key to exit..."
    invoke ExitProcess, 0

main ENDP
end main

```

mylib

```

.586p
.model flat, stdcall
option casemap: none

.code
shaker_sort PROC uses eax ebx ecx edx esi edi, arrLen:DWORD, arrAddr:DWORD
    LOCAL left:DWORD
    LOCAL right:DWORD
    LOCAL swapped:DWORD

    mov left, 0
    mov eax, arrLen
    dec eax
    mov right, eax
    mov swapped, 1

sort_loop:
    cmp swapped, 0
    je sort_done

    mov swapped, 0

    mov ecx, left
    mov edi, arrAddr

left_to_right:
    cmp ecx, right
    jge right_to_left_start

    mov eax, [edi + ecx*4]
    mov edx, [edi + ecx*4 + 4]

    cmp eax, edx
    jge no_swap_right

    mov [edi + ecx*4], edx
    mov [edi + ecx*4 + 4], eax
    mov swapped, 1

no_swap_right:
    inc ecx
    jmp left_to_right

right_to_left_start:
    cmp swapped, 0
    je sort_done

    dec right

    mov ecx, right

```

```

    mov edi, arrAddr

right_to_left:
    cmp ecx, left
    jle update_left

    mov eax, [edi + ecx*4]
    mov edx, [edi + ecx*4 - 4]

    cmp edx, eax
    jge no_swap_left

    mov [edi + ecx*4], edx
    mov [edi + ecx*4 - 4], eax
    mov swapped, 1

no_swap_left:
    dec ecx
    jmp right_to_left

update_left:
    inc left
    jmp sort_loop

sort_done:
    ret
shaker_sort ENDP

end

```

Способ сборки

```

\masm32\bin\ml.exe /c /coff @list.rsp
\masm32\bin\link -lib *.obj /out:mylib.lib

```

в list.rsp указано название файла с libой

Вывод

я научился собирать библиотеки для masm32